注释变换器

2018年4月3日

————————-

现在这篇博客文章有了更新 (http://harvardnlp.github.io/annotated-transformer)版本，适用于现代PyTorch。

————————-

```
from IPython.display import Image
Image(filename='images/aiayn.png')
```

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

《关注全是你》中的变形金刚 “需求” (https://arxiv.org/abs/1706.03762)一直是很多人心头的念头 在过去一年里。除了大幅提升翻译质量， 它为许多其他自然语言处理任务提供了新的架构。论文本身是 写得非常清楚，但普遍看法是它相当 很难正确实现。

在这篇文章中，我呈现一篇“带注释”的论文版本，形式为 逐行实现。我已经重新排序并删除了一些部分 原文并贯穿全文添加评论。这份文件本身就是一份工作文件 笔记本，应该是一个完全可用的实现。总共有400行库代码，可在4块GPU上每秒处理27,000个令牌。

要跟进，你首先需要安装PyTorch (http://pytorch.org/)。完整的笔记本也 可在GitHub (https://github.com/harvardnlp/annotated-transformer)或 谷歌 Colab (https://drive.google.com/file/d/1xQXSv6mtAOLXxEMi8RvaW8TW-7bvYBDF/view?usp=sh aring) 搭配免费显卡。

请注意，这仅是研究人员和感兴趣开发者的起点。 这里的代码大量基于我们的 OpenNMT (http://opennmt.net) 包。 （如果有帮助，欢迎引用。）关于其他全权服务 模型的实现检验为 Tensor2Tensor (https://github.com/tensorflow/tensor2tensor)（tensorflow）和 Sockeye (https://github.com/awslabs/sockeye)（mxnet）。

- 亚历山大·拉什（@harvardnlp (https://twitter.com/harvardnlp) 或 srush@seas.harvard.edu），并得到阮文森和纪尧姆·克莱因的协助

初赛

```
# !pip install http://download.pytorch.org/whl/cu80/torch-0.3.0.post4-cp36-cp36m-linux_x86_64.whl (http://download.pytorch.org/whl/cu80/torch-0.3.0.post4-cp36-cp36m-linux_x86_64.whl) numpy matplotlib spacy torchtext seaborn
```

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import math, copy, time
from torch.autograd import Variable
import matplotlib.pyplot as plt
import seaborn
seaborn.set_context(context="talk")
%matplotlib inline
```

目录

- 初赛
- 背景
- 模型架构
  - 编码器和解码堆栈
    - 编码器
    - 解码器
    - 注意
    - 注意力在我们模型中的应用
  - 按位置划分的前馈网络
  - 嵌入与Softmax
  - 位置编码
  - 完整模型
- 培训
  - 批次与掩蔽
  - 训练环路
  - 训练数据与批处理
  - 硬件与赛程
  - 优化器
  - 正则化
    - 标签平滑
- 第一个例子
  - 合成数据
  - 损失计算
  - 贪婪解码
- 一个真实世界的例子
  - 数据加载
  - 迭代器
  - 多GPU训练
  - 系统培训
- 附加组件：BPE、搜索、平均
- 选举结果
  - 注意力可视化
- 结论

> *我的评论已被引用。正文全都来自论文本身。*

背景

减少顺序计算的目标也构成了 扩展神经GPU、ByteNet和ConvS2S，均使用卷积神经 网络作为基本构建模块，并行计算隐藏表示 适用于所有输入和输出位置。在这些模型中，操作次数 需要将两个任意输入或输出位置的信号关联起来，增长于 对于ConvS2S，位置之间的距离为线性距离，对于对数为 字节网。这使得学习远距离依赖关系变得更加困难 职位。在变换器中，这个被简化为常数 尽管由于平均化，有效分辨率会降低 注意力加权的位置，我们用多头注意力来抵消这种效应。

自我注意力，有时也称为内注意，是一种注意力机制 通过关联单个序列的不同位置来计算 序列的表示。自我关注已被成功应用于 任务种类繁多，包括阅读理解、抽象摘要， 文本蕴涵与学习任务无关句子表示。结束—— 终端记忆网络基于循环注意力机制，而非 序列比对复发，且已被证明在简单的—— 语言问题解答和语言建模任务。

然而，据我们所知，变换器是第一个转导器 完全依靠自我关注来计算输入的表示的模型 并且输出时不使用序列对齐的RNN或卷积。

模型架构

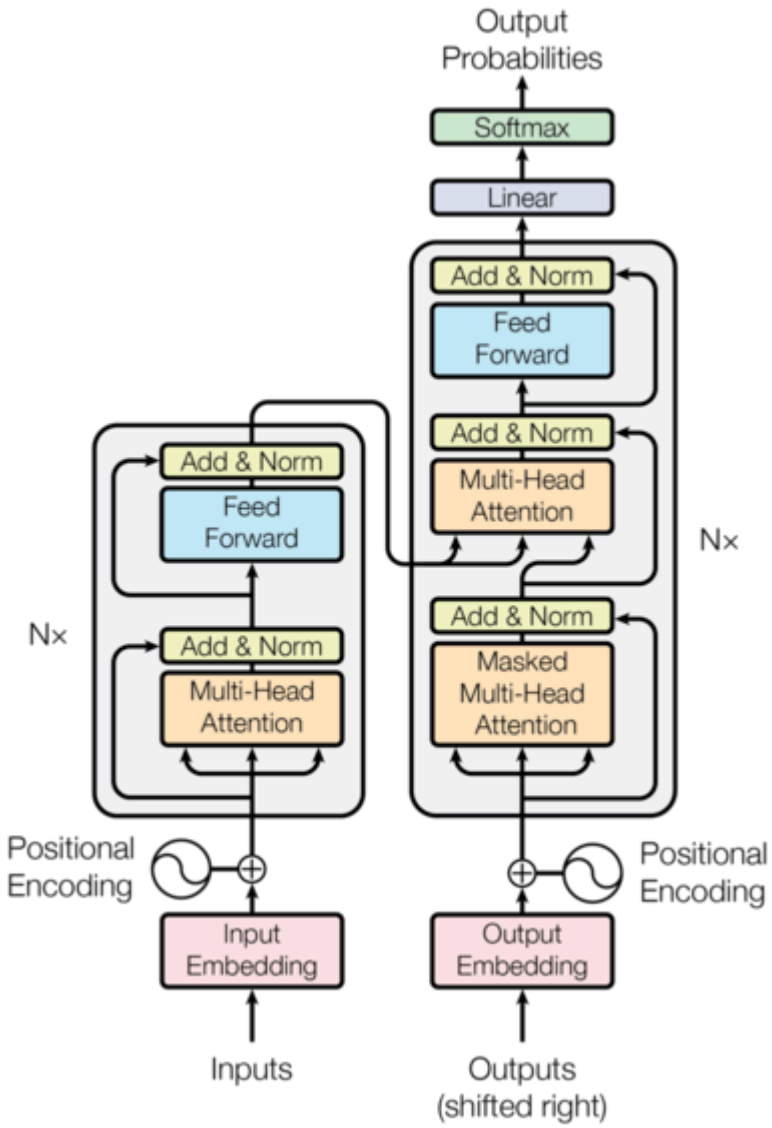
大多数竞争性的神经序列转导模型都采用编码-解码器 结构（CITE）。 (https://arxiv.org/abs/1409.0473)这里，编码器映射 输入符号表示序列$(x_1, \ldots, x_n)$映射为 连续表示$\mathbf{z} = (z_1, \ldots, z_n)$.给定$\mathbf{z}$, 解码器随后生成输出序列$(y_1, \ldots, y_m)$象征一 一次一个元素。每一步模型都是自回归（引用），(https://arxiv.org/abs/1308.0850)消耗之前生成的 在生成下一个符号时作为额外输入。

```
class EncoderDecoder(nn.Module):
    """
    A standard Encoder-Decoder architecture. Base for this and many
    other models.
    """
    def __init__(self, encoder, decoder, src_embed, tgt_embed, generator):
        super(EncoderDecoder, self).__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.src_embed = src_embed
        self.tgt_embed = tgt_embed
        self.generator = generator

    def forward(self, src, tgt, src_mask, tgt_mask):
        "Take in and process masked src and target sequences."
        return self.decode(self.encode(src, src_mask), src_mask,
                           tgt, tgt_mask)

    def encode(self, src, src_mask):
        return self.encoder(self.src_embed(src), src_mask)

    def decode(self, memory, src_mask, tgt, tgt_mask):
        return self.decoder(self.tgt_embed(tgt), memory, src_mask, tgt_mask)
```

```
class Generator(nn.Module):
    "Define standard linear + softmax generation step."
    def __init__(self, d_model, vocab):
        super(Generator, self).__init__()
        self.proj = nn.Linear(d_model, vocab)

    def forward(self, x):
        return F.log_softmax(self.proj(x), dim=-1)
```

变压器采用了这种整体架构，采用叠加自我关注 以及编码器和解码器的点位全连通层，如下所示 分别出现在图1的左半部分和右半部分。

```
Image(filename='images/ModalNet-21.png')
```



编码器和解码堆栈

编码器

编码器由一组$N = 6$一模一样的层次。

```
def clones(module, N):
    "Produce N identical layers."
    return nn.ModuleList([copy.deepcopy(module) for _ in range(N)])
```

```
class Encoder(nn.Module):
    "Core encoder is a stack of N layers"
    def __init__(self, layer, N):
        super(Encoder, self).__init__()
        self.layers = clones(layer, N)
        self.norm = LayerNorm(layer.size)

    def forward(self, x, mask):
        "Pass the input (and mask) through each layer in turn."
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)
```

我们在两个子层之间使用残差连接（cite）， (https://arxiv.org/abs/1512.03385)然后进行层归一化（cite）。(https://arxiv.org/abs/1607.06450)

```
class LayerNorm(nn.Module):
    "Construct a layernorm module (See citation for details)."
    def __init__(self, features, eps=1e-6):
        super(LayerNorm, self).__init__()
        self.a_2 = nn.Parameter(torch.ones(features))
        self.b_2 = nn.Parameter(torch.zeros(features))
        self.eps = eps

    def forward(self, x):
        mean = x.mean(-1, keepdim=True)
        std = x.std(-1, keepdim=True)
        return self.a_2 * (x - mean) / (std + self.eps) + self.b_2
```

也就是说，每个子层的输出为$\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$， 其中$\mathrm{Sublayer}(x)$是实现的函数 由子层本身。我们对每个输出应用dropout（引用） (http://jmlr.org/papers/v15/srivastava14a.html) 子层，在将其加入子层输入并归一化之前。

为了促进这些残留连接，模型中的所有子层也同样如此 作为嵌入层，产生维度为 的输出$d_{\text{模型}} = 512.$

```
class SublayerConnection(nn.Module):
    """
    A residual connection followed by a layer norm.
    Note for code simplicity the norm is first as opposed to last.
    """
    def __init__(self, size, dropout):
        super(SublayerConnection, self).__init__()
        self.norm = LayerNorm(size)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, sublayer):
        "Apply residual connection to any sublayer with the same size."
        return x + self.dropout(sublayer(self.norm(x)))
```

每层有两个子层。第一种是多头的自我关注 第二个是简单的、按位置完全连接的供给—— 前进网络。

```
class EncoderLayer(nn.Module):
    "Encoder is made up of self-attn and feed forward (defined below)"
    def __init__(self, size, self_attn, feed_forward, dropout):
        super(EncoderLayer, self).__init__()
        self.self_attn = self_attn
        self.feed_forward = feed_forward
        self.sublayer = clones(SublayerConnection(size, dropout), 2)
        self.size = size

    def forward(self, x, mask):
        "Follow Figure 1 (left) for connections."
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, mask))
        return self.sublayer[1](x, self.feed_forward)
```

解码器

解码器还由一组$N = 6$一模一样的层次。

```
class Decoder(nn.Module):
    "Generic N layer decoder with masking."
    def __init__(self, layer, N):
        super(Decoder, self).__init__()
        self.layers = clones(layer, N)
        self.norm = LayerNorm(layer.size)

    def forward(self, x, memory, src_mask, tgt_mask):
        for layer in self.layers:
            x = layer(x, memory, src_mask, tgt_mask)
        return self.norm(x)
```

除了每个编码层的两个子层外，解码器还插入 第三子层，对输出进行多头关注， 编码器栈。类似于编码器，我们会在周围使用残余连接 每个子层，然后是层规范化。

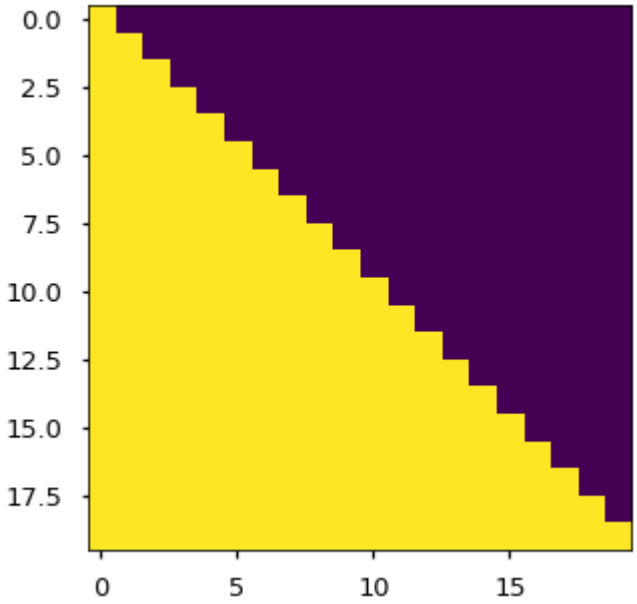
```
class DecoderLayer(nn.Module):
    "Decoder is made of self-attn, src-attn, and feed forward (defined below)"
    def __init__(self, size, self_attn, src_attn, feed_forward, dropout):
        super(DecoderLayer, self).__init__()
        self.size = size
        self.self_attn = self_attn
        self.src_attn = src_attn
        self.feed_forward = feed_forward
        self.sublayer = clones(SublayerConnection(size, dropout), 3)

    def forward(self, x, memory, src_mask, tgt_mask):
        "Follow Figure 1 (right) for connections."
        m = memory
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, tgt_mask))
        x = self.sublayer[1](x, lambda x: self.src_attn(x, m, m, src_mask))
        return self.sublayer[2](x, self.feed_forward)
```

我们还修改了解码器栈中的自注意子层，以防止 从助理职位到后续职位。这种掩饰，结合起来 输出嵌入偏移一个位置，确保了 位置预测i只能依赖于已知的输出 低于i.

```
def subsequent_mask(size):
    "Mask out subsequent positions."
    attn_shape = (1, size, size)
    subsequent_mask = np.triu(np.ones(attn_shape), k=1).astype('uint8')
    return torch.from_numpy(subsequent_mask) == 0
```

> *注意面罩下方显示每个tgt词（行）允许的位置 看看（列）。在 训练。*

```
plt.figure(figsize=(5,5))
plt.imshow(subsequent_mask(20)[0])
None
```

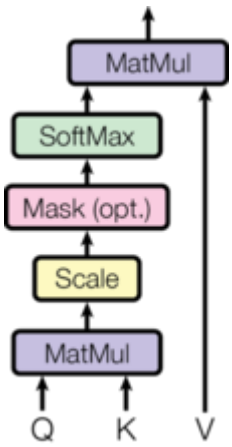


注意

注意力函数可以描述为映射查询和一组键值 与输出对对，其中查询、键、值和输出都是向量。 输出以加权和的形式计算，其中权重为 每个值由查询的兼容性函数计算，满足 对应的密钥。

我们称我们的特别关注点为“缩放点积关注”。输入 由维数为 的查询和键组成d_k，以及维数的值d_v. 我们计算查询与所有键的点积，并对每个键除以$\sqrt{d_k}$，并应用软最大值函数以获得这些值的权重。

```
Image(filename='images/ModalNet-19.png')
```



实际上，我们计算一组查询的注意力函数 同时，被压缩成一个矩阵Q.键和值分别为 也被打包成矩阵K以及V.我们计算 的矩阵 输出为：

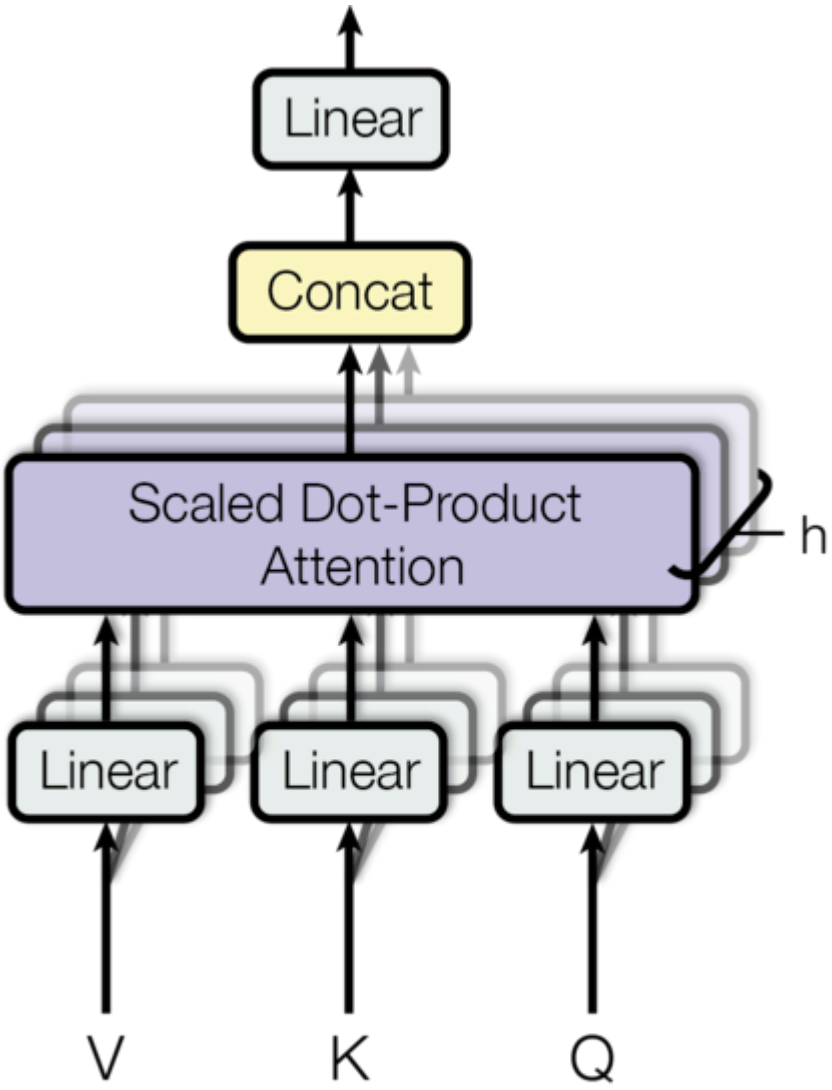
$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

```
def attention(query, key, value, mask=None, dropout=None):
    "Compute 'Scaled Dot Product Attention'"
    d_k = query.size(-1)
    scores = torch.matmul(query, key.transpose(-2, -1)) \
             / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)
    p_attn = F.softmax(scores, dim = -1)
    if dropout is not None:
        p_attn = dropout(p_attn)
    return torch.matmul(p_attn, value), p_attn
```

最常用的两个注意力函数是加法注意力（cite） (https://arxiv.org/abs/1409.0473)和点积（乘法） 注意。点积注意力与我们的算法相同，唯一区别在于 缩放因子为$\frac{1}{\sqrt{d_k}}$.加法注意力计算 兼容性功能使用带有单一隐藏层的前馈网络。 虽然两者在理论复杂度上相似，但点积注意力是 在实际操作中，速度更快、空间更高效，因为它可以实现 使用高度优化的矩阵乘法代码。

而对于小值d_k这两个机制的工作原理相似，都是加法的 注意力在不进行缩放的情况下，在较大的值下优于点积注意力d_k （引用）。 (https://arxiv.org/abs/1703.03906)我们怀疑这点 值d_k点积的大小会增加，推动软极大值 作用于梯度极小的区域（说明原因） 点积变大，假设分量为q以及k是 均值为0以及方差1.然后是他们的点 产品，$q \cdot k = \sum_{i=1}^{d_k} q_i k_i$，均值为$0$以及方差$d_k$.).为抵消这一效应，我们将点积按比例$\frac{1}{\sqrt{d_k}}$.

```
Image(filename='images/ModalNet-20.png')
```



多头注意力使模型能够共同关注以下信息 不同位置的表示子空间。一枚单曲 注意，平均化会抑制这种情况。

$\mathrm{MultiHead}(Q,K,V)=\mathrm{Concat}(\mathrm{head_1},\ldots,\mathrm{head_h})W^O$

其中 $\mathrm{head_i}=\mathrm{Attention}(QW_i^Q,KW_i^K,VW_i^V)$

其中投影为参数矩阵$W_i^Q\in\mathbb{R}^{d_{模型}\times d_k}$,$W_i^K\in\mathbb{R}^{d_{模型}\times d_k}$,$W_i^V\in\mathbb{R}^{d_{模型}\times d_v}$以及$W^O\in\mathbb{R}^{hd_v\times d_{模型}}$. 在这项工作中，我们聘用的$h=8$平行注意力层，也就是头部。对于每个 我们用这些$d_k=d_v=d_{模型}/h=64$.由于的约化维数 每个头的总计算成本与单头相似 全维度的专注。

```
class MultiHeadedAttention(nn.Module):
    def __init__(self, h, d_model, dropout=0.1):
        "Take in model size and number of heads."
        super(MultiHeadedAttention, self).__init__()
        assert d_model % h == 0
        # We assume d_v always equals d_k
        self.d_k = d_model // h
        self.h = h
        self.linears = clones(nn.Linear(d_model, d_model), 4)
        self.attn = None
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, query, key, value, mask=None):
        "Implements Figure 2"
        if mask is not None:
            # Same mask applied to all h heads.
            mask = mask.unsqueeze(1)
        nbatches = query.size(0)

        # 1) Do all the linear projections in batch from d_model => h x d_k
        query, key, value = \
            [l(x).view(nbatches, -1, self.h, self.d_k).transpose(1, 2)
             for l, x in zip(self.linears, (query, key, value))]

        # 2) Apply attention on all the projected vectors in batch.
        x, self.attn = attention(query, key, value, mask=mask,
                                 dropout=self.dropout)

        # 3) "Concat" using a view and apply a final linear.
        x = x.transpose(1, 2).contiguous() \
             .view(nbatches, -1, self.h * self.d_k)
        return self.linears[-1](x)
```

注意力在我们模型中的应用

变压金刚以三种不同方式使用多头注意力： 1）在“编码器-解码器注意”层中，查询来自前一层 解码层，内存键和值来自 编码器。这使得解码器中的所有位置都能全域关注 输入序列中的位置。这模拟了典型的编码-解码器 序列到序列模型中的注意力机制，如（CITE）。 (https://arxiv.org/abs/1609.08144)

2）编码器包含自注意层。在自我关注层中，所有 键、值和查询都来自同一地方，在这里是输出 编码器中前一层的。编码器中的每个位置都可以参与 返回编码器前一层的所有位置。

3）类似地，解码器中的自关注层允许每个位置 解码器负责处理解码器中所有位置，包括 位置。我们需要防止解码器中向左的信息流，以实现 保持自回归性质。我们将它实现在缩放点内—— 通过遮蔽出（设置为 $-\infty$）所有输入中的值 对应于非法连接的Softmax。

按位置划分的前馈网络

除了注意力子层外，我们编码器中的每一层和 解码器包含一个完全连接的前馈网络，应用于 每个位置分别且完全相同的位置。这由两个线性组成 变形中介于ReLU激活。

$$\mathrm{FFN}(x) = \text{马克斯}(0, xW_1 + b_1)W_2 + b_2$$

虽然线性变换在不同位置上相同，但它们 每层使用不同的参数。换句话说 作为两个核大小为1的卷积。输入和输出的维度 是$d_{模型} = 512$，内层具有维度$d_{ff} = 2048$.

```
class PositionwiseFeedForward(nn.Module):
    "Implements FFN equation."
    def __init__(self, d_model, d_ff, dropout=0.1):
        super(PositionwiseFeedForward, self).__init__()
        self.w_1 = nn.Linear(d_model, d_ff)
        self.w_2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        return self.w_2(self.dropout(F.relu(self.w_1(x))))
```

嵌入与Softmax

与其他序列转导模型类似，我们使用学习到的嵌入 将输入标记和输出标记转换为维数为$d_{模型}$.我们还使用通常学习的线性变换和 SoftMax函数将译码器输出转换为预测的下一个令牌 概率。在我们的模型中，两者共享相同的权重矩阵 嵌入层和pre-softmax线性变换，类似于（CITE） (https://arxiv.org/abs/1608.05859)。在嵌入层中，我们会乘数 这些权重由$\sqrt{d_{模型}}$.

```
class Embeddings(nn.Module):
    def __init__(self, d_model, vocab):
        super(Embeddings, self).__init__()
        self.lut = nn.Embedding(vocab, d_model)
        self.d_model = d_model

    def forward(self, x):
        return self.lut(x) * math.sqrt(self.d_model)
```

位置编码

由于我们的模型不包含递归和卷积，因此 模型 为了利用序列的顺序，我们必须注入一些信息 关于序列中标记的相对或绝对位置。对此 最后，我们在输入嵌入底部添加“位置编码” 编码器和解码堆栈。位置编码的维度相同$d_{模型}$作为嵌入，使两者可以相加。有 多种位置编码选择，学习型和固定型（引用）。(https://arxiv.org/pdf/1705.03122.pdf)

在本工作中，我们使用不同频率的正弦和余弦函数：$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{模型}})$

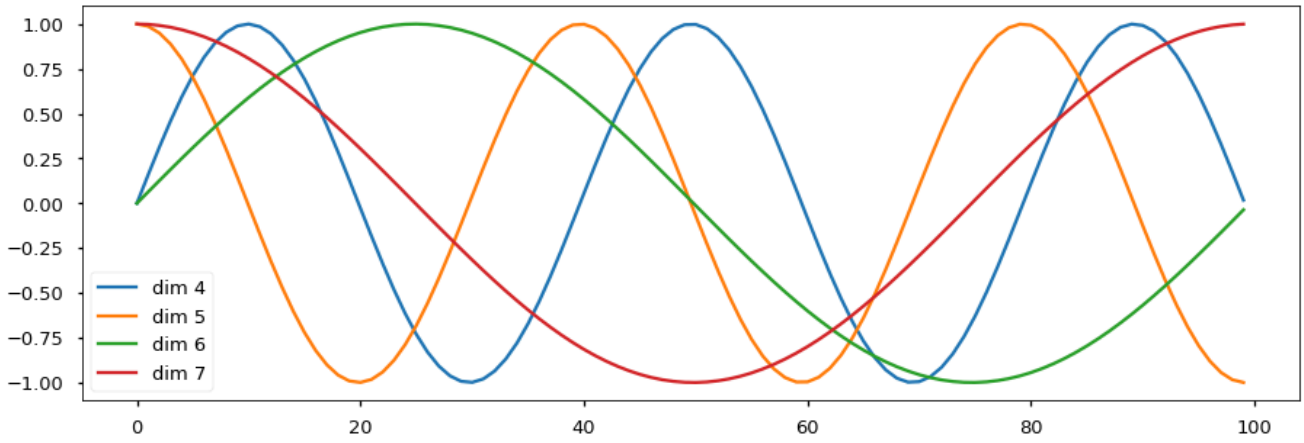
$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{模型}})$其中$pos$是位置，且$i$是维数。也就是说，每个维度 其中的位置编码对应于正弦波。波长形成 几何级数从2π到$10000 \cdot 2\pi$.我们选择了这个函数 因为我们假设它能让模型轻松学习关注 相对位置，因为对于任意固定偏移k,PE_{pos+k}可以是 表示为 的线性函数PE_{pos}.

此外，我们还对嵌入和位置的和应用了退出 编码器和解码器堆栈中的编码。对于基础模型，我们使用 速率$P_{drop} = 0.1$.

```
class PositionalEncoding(nn.Module):
    "Implement the PE function."
    def __init__(self, d_model, dropout, max_len=5000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

        # Compute the positional encodings once in log space.
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) *
                             -(math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + Variable(self.pe[:, :x.size(1)],
                         requires_grad=False)
        return self.dropout(x)
```

> *在位置编码下方，会根据位置加入一个正弦波。该 波的频率和偏移量在每个维度上都不同。*

```
plt.figure(figsize=(15, 5))
pe = PositionalEncoding(20, 0)
y = pe.forward(Variable(torch.zeros(1, 100, 20)))
plt.plot(np.arange(100), y[0, :, 4:8].data.numpy())
plt.legend(["dim %d"%p for p in [4,5,6,7]])
None
```



我们还尝试使用学习到的位置嵌入（cite）， (https://arxiv.org/pdf/1705.03122.pdf)发现两者 各版本的结果几乎完全相同。我们选择了正弦波版本 因为它可能允许模型外推到比 训练时遇到的那些。

完整模型

> *这里我们定义一个函数，输入超参数并生成完整模型。*

```
def make_model(src_vocab, tgt_vocab, N=6,
               d_model=512, d_ff=2048, h=8, dropout=0.1):
    "Helper: Construct a model from hyperparameters."
    c = copy.deepcopy
    attn = MultiHeadedAttention(h, d_model)
    ff = PositionwiseFeedForward(d_model, d_ff, dropout)
    position = PositionalEncoding(d_model, dropout)
    model = EncoderDecoder(
        Encoder(EncoderLayer(d_model, c(attn), c(ff), dropout), N),
        Decoder(DecoderLayer(d_model, c(attn), c(attn),
                             c(ff), dropout), N),
        nn.Sequential(Embeddings(d_model, src_vocab), c(position)),
        nn.Sequential(Embeddings(d_model, tgt_vocab), c(position)),
        Generator(d_model, tgt_vocab))

    # This was important from their code.
    # Initialize parameters with Glorot / fan_avg.
    for p in model.parameters():
        if p.dim() > 1:
            nn.init.xavier_uniform(p)
    return model
```

```
# Small example model.
tmp_model = make_model(10, 10, 2)
None
```

培训

本节描述了我们模型的训练方案。

> *我们稍作插曲介绍一些工具 需要训练标准编码器解码器模型。首先我们定义一个批处理对象 它包含SRC和目标句子用于训练，同时构建 面具。*

批次与掩蔽

```
class Batch:
    "Object for holding a batch of data with mask during training."
    def __init__(self, src, trg=None, pad=0):
        self.src = src
        self.src_mask = (src != pad).unsqueeze(-2)
        if trg is not None:
            self.trg = trg[:, :-1]
            self.trg_y = trg[:, 1:]
            self.trg_mask = \
                self.make_std_mask(self.trg, pad)
            self.ntokens = (self.trg_y != pad).data.sum()

    @staticmethod
    def make_std_mask(tgt, pad):
        "Create a mask to hide padding and future words."
        tgt_mask = (tgt != pad).unsqueeze(-2)
        tgt_mask = tgt_mask & Variable(
            subsequent_mask(tgt.size(-1)).type_as(tgt_mask.data))
        return tgt_mask
```

> *接下来我们创建一个通用的训练和评分函数来跟踪减重情况。 我们传递一个通用的损耗计算函数，它也处理参数更新。*

训练环路

```
def run_epoch(data_iter, model, loss_compute):
    "Standard Training and Logging Function"
    start = time.time()
    total_tokens = 0
    total_loss = 0
    tokens = 0
    for i, batch in enumerate(data_iter):
        out = model.forward(batch.src, batch.trg,
                            batch.src_mask, batch.trg_mask)
        loss = loss_compute(out, batch.trg_y, batch.ntokens)
        total_loss += loss
        total_tokens += batch.ntokens
        tokens += batch.ntokens
        if i % 50 == 1:
            elapsed = time.time() - start
            print("Epoch Step: %d Loss: %f Tokens per Sec: %f" %
                    (i, loss / batch.ntokens, tokens / elapsed))
            start = time.time()
            tokens = 0
    return total_loss / total_tokens
```

训练数据与批处理

我们基于标准WMT 2014英德数据集进行训练，内容约为 450万对句子。句子采用字节对编码， 该系统共有约37000个代币的源-目标词汇。关于英语—— 法语方面，我们使用了显著更大的WMT 2014英法数据集由3600万句子组成，将标记拆分为32000字的词汇表。

句子对按近似序列长度批量分类。每人 训练批包含一组约25000对的句子对 源代币和25000个目标代币。

> *我们将使用火炬文本进行批量处理。下面会对此有更详细的讨论。 这里我们用 torchtext 函数创建批量，确保批量大小 填充到最大批次大小不会超过阈值（如果有8个，则是25000 GPU）。*

```
global max_src_in_batch, max_tgt_in_batch
def batch_size_fn(new, count, sofar):
    "Keep augmenting batch and calculate total number of tokens + padding."
    global max_src_in_batch, max_tgt_in_batch
    if count == 1:
        max_src_in_batch = 0
        max_tgt_in_batch = 0
    max_src_in_batch = max(max_src_in_batch,  len(new.src))
    max_tgt_in_batch = max(max_tgt_in_batch,  len(new.trg) + 2)
    src_elements = count * max_src_in_batch
    tgt_elements = count * max_tgt_in_batch
    return max(src_elements, tgt_elements)
```

硬件与赛程

我们在一台配备8块NVIDIA P100 GPU的机器上训练模型。为了我们的基地 使用论文中描述的超参数模型，每次训练 步速大约花了0.4秒。我们训练了总共10万个基础模型 要么是12小时，要么是每一步。对于我们的大型模型，步进时间是1.0秒。大事 模型训练了30万步（3.5天）。

优化器

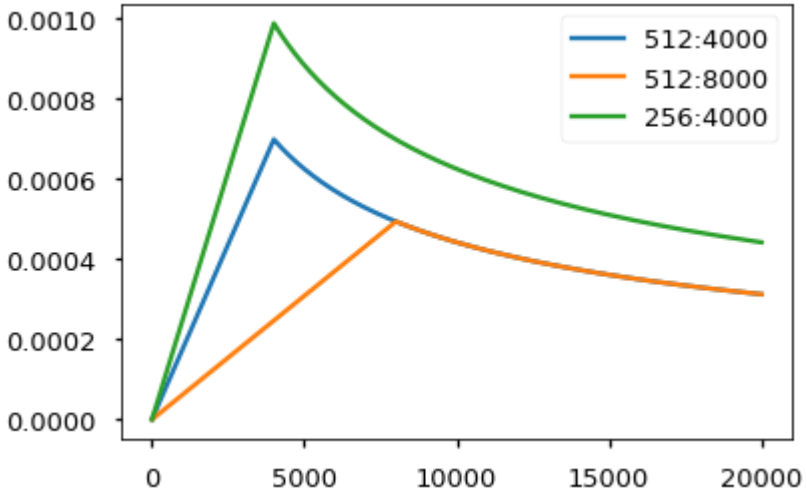
我们使用了Adam优化器（引用）， (https://arxiv.org/abs/1412.6980)$\beta_1 = 0.9$,$\beta_2 = 0.98$以及$\varepsilon = 10^{-9}$.我们多样地学习 训练期间的评分，公式如下：
$lrate = d_{\text{模型}}^{-0.5} \cdot \text{明}(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$这对应于第一个学习率线性提升$warmup_steps$训练步数，之后按比例减少 步数的反平方根。我们用$warmup_steps = 4000$.

> *注意：这部分非常重要。需要用这个模型的设置来训练。*

```
class NoamOpt:
    "Optim wrapper that implements rate."
    def __init__(self, model_size, factor, warmup, optimizer):
        self.optimizer = optimizer
        self._step = 0
        self.warmup = warmup
        self.factor = factor
        self.model_size = model_size
        self._rate = 0

    def step(self):
        "Update parameters and rate"
        self._step += 1
        rate = self.rate()
        for p in self.optimizer.param_groups:
            p['lr'] = rate
        self._rate = rate
        self.optimizer.step()

    def rate(self, step = None):
        "Implement `lrate` above"
        if step is None:
            step = self._step
        return self.factor * \
            (self.model_size ** (-0.5) *
            min(step ** (-0.5), step * self.warmup ** (-1.5)))

def get_std_opt(model):
    return NoamOpt(model.src_embed[0].d_model, 2, 4000,
            torch.optim.Adam(model.parameters(), lr=0, betas=(0.9, 0.98), eps=1e-9))
```

> *该模型在不同模型尺寸和 优化超参数。*

```
# Three settings of the lrate hyperparameters.
opts = [NoamOpt(512, 1, 4000, None),
        NoamOpt(512, 1, 8000, None),
        NoamOpt(256, 1, 4000, None)]
plt.plot(np.arange(1, 20000), [[opt.rate(i) for opt in opts] for i in range(1, 20000)])
plt.legend(["512:4000", "512:8000", "256:4000"])
None
```



正则化

标签平滑

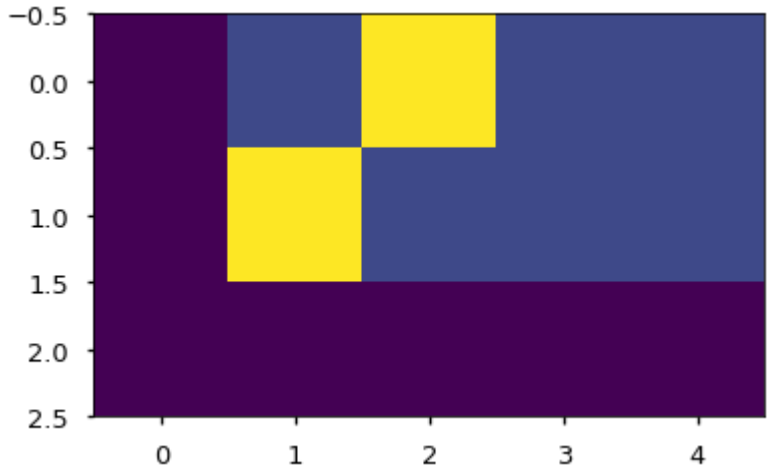
培训期间，我们采用了价值的标签平滑处理$\varepsilon_{ls} = 0.1$ （引用）。 (https://arxiv.org/abs/1512.00567)这会让困惑度受损，因为模型 学会更不确定，但能提高准确率和BLEU分数。

> *我们用KL div损耗实现标签平滑。而不是用一热器 目标分布，我们创建一个分布，使得 正确的词语以及其他弥撒分布在 词汇量。* `confidence` `smoothing`

```
class LabelSmoothing(nn.Module):
    "Implement label smoothing."
    def __init__(self, size, padding_idx, smoothing=0.0):
        super(LabelSmoothing, self).__init__()
        self.criterion = nn.KLDivLoss(size_average=False)
        self.padding_idx = padding_idx
        self.confidence = 1.0 - smoothing
        self.smoothing = smoothing
        self.size = size
        self.true_dist = None

    def forward(self, x, target):
        assert x.size(1) == self.size
        true_dist = x.data.clone()
        true_dist.fill_(self.smoothing / (self.size - 2))
        true_dist.scatter_(1, target.data.unsqueeze(1), self.confidence)
        true_dist[:, self.padding_idx] = 0
        mask = torch.nonzero(target.data == self.padding_idx)
        if mask.dim() > 0:
            true_dist.index_fill_(0, mask.squeeze(), 0.0)
        self.true_dist = true_dist
        return self.criterion(x, Variable(true_dist, requires_grad=False))
```

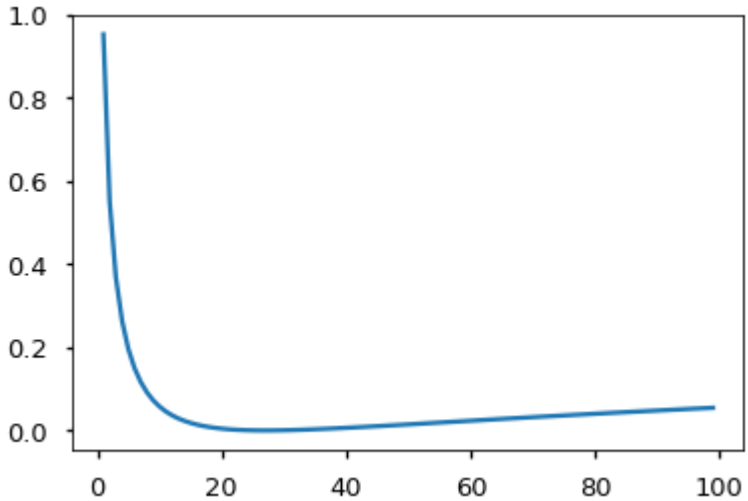
> *这里我们可以看到质量如何分布到基于单词的例子 信任。*

```
# Example of label smoothing.
crit = LabelSmoothing(5, 0, 0.4)
predict = torch.FloatTensor([[0, 0.2, 0.7, 0.1, 0],
                             [0, 0.2, 0.7, 0.1, 0],
                             [0, 0.2, 0.7, 0.1, 0]])
v = crit(Variable(predict.log()),
         Variable(torch.LongTensor([2, 1, 0])))

# Show the target distributions expected by the system.
plt.imshow(crit.true_dist)
None
```



> *标签平滑实际上会开始惩罚模型，如果它变得非常 对某个选择充满信心。*

```
crit = LabelSmoothing(5, 0, 0.1)
def loss(x):
    d = x + 3 * 1
    predict = torch.FloatTensor([[0, x / d, 1 / d, 1 / d, 1 / d],
                                 ])
    #print(predict)
    return crit(Variable(predict.log()),
                Variable(torch.LongTensor([1]))).data[0]
plt.plot(np.arange(1, 100), [loss(x) for x in range(1, 100)])
None
```



第一个例子

> *我们可以先尝试一个简单的复制任务。给定一组随机输入 从少量词汇中获得符号，目标是生成相同的符号 符号。*

合成数据

```
def data_gen(V, batch, nbatches):
    "Generate random data for a src-tgt copy task."
    for i in range(nbatches):
        data = torch.from_numpy(np.random.randint(1, V, size=(batch, 10)))
        data[:, 0] = 1
        src = Variable(data, requires_grad=False)
        tgt = Variable(data, requires_grad=False)
        yield Batch(src, tgt, 0)
```

损失计算

```
class SimpleLossCompute:
    "A simple loss compute and train function."
    def __init__(self, generator, criterion, opt=None):
        self.generator = generator
        self.criterion = criterion
        self.opt = opt

    def __call__(self, x, y, norm):
        x = self.generator(x)
        loss = self.criterion(x.contiguous().view(-1, x.size(-1)),
                              y.contiguous().view(-1)) / norm
        loss.backward()
        if self.opt is not None:
            self.opt.step()
            self.opt.optimizer.zero_grad()
        return loss.data[0] * norm
```

贪婪解码

```
# Train the simple copy task.
V = 11
criterion = LabelSmoothing(size=V, padding_idx=0, smoothing=0.0)
model = make_model(V, V, N=2)
model_opt = NoamOpt(model.src_embed[0].d_model, 1, 400,
        torch.optim.Adam(model.parameters(), lr=0, betas=(0.9, 0.98), eps=1e-9))

for epoch in range(10):
    model.train()
    run_epoch(data_gen(V, 30, 20), model,
              SimpleLossCompute(model.generator, criterion, model_opt))
    model.eval()
    print(run_epoch(data_gen(V, 30, 5), model,
                    SimpleLossCompute(model.generator, criterion, None)))
```

```
Epoch Step: 1 Loss: 3.023465 Tokens per Sec: 403.074173
Epoch Step: 1 Loss: 1.920030 Tokens per Sec: 641.689380
1.9274832487106324
Epoch Step: 1 Loss: 1.940011 Tokens per Sec: 432.003378
Epoch Step: 1 Loss: 1.699767 Tokens per Sec: 641.979665
1.657595729827881
Epoch Step: 1 Loss: 1.860276 Tokens per Sec: 433.320240
Epoch Step: 1 Loss: 1.546011 Tokens per Sec: 640.537198
1.4888023376464843
Epoch Step: 1 Loss: 1.682198 Tokens per Sec: 432.092305
Epoch Step: 1 Loss: 1.313169 Tokens per Sec: 639.441857
1.3485562801361084
Epoch Step: 1 Loss: 1.278768 Tokens per Sec: 433.568756
Epoch Step: 1 Loss: 1.062384 Tokens per Sec: 642.542067
0.9853351473808288
Epoch Step: 1 Loss: 1.269471 Tokens per Sec: 433.388727
Epoch Step: 1 Loss: 0.590709 Tokens per Sec: 642.862135
0.5686767101287842
Epoch Step: 1 Loss: 0.997076 Tokens per Sec: 433.009746
Epoch Step: 1 Loss: 0.343118 Tokens per Sec: 642.288427
0.34273059368133546
Epoch Step: 1 Loss: 0.459483 Tokens per Sec: 434.594030
Epoch Step: 1 Loss: 0.290385 Tokens per Sec: 642.519464
0.2612409472465515
Epoch Step: 1 Loss: 1.031042 Tokens per Sec: 434.557008
Epoch Step: 1 Loss: 0.437069 Tokens per Sec: 643.630322
0.4323212027549744
Epoch Step: 1 Loss: 0.617165 Tokens per Sec: 436.652626
Epoch Step: 1 Loss: 0.258793 Tokens per Sec: 644.372296
0.27331129014492034
```

> *该码通过贪婪解码为简化而预测翻译。*

```
def greedy_decode(model, src, src_mask, max_len, start_symbol):
    memory = model.encode(src, src_mask)
    ys = torch.ones(1, 1).fill_(start_symbol).type_as(src.data)
    for i in range(max_len-1):
        out = model.decode(memory, src_mask,
                           Variable(ys),
                           Variable(subsequent_mask(ys.size(1))
                                    .type_as(src.data)))
        prob = model.generator(out[:, -1])
        _, next_word = torch.max(prob, dim = 1)
        next_word = next_word.data[0]
        ys = torch.cat([ys,
                        torch.ones(1, 1).type_as(src.data).fill_(next_word)], dim=1)
    return ys

model.eval()
src = Variable(torch.LongTensor([[1,2,3,4,5,6,7,8,9,10]]) )
src_mask = Variable(torch.ones(1, 1, 10) )
print(greedy_decode(model, src, src_mask, max_len=10, start_symbol=1))
```

```
    1     2     3     4     5     6     7     8     9    10
[torch.LongTensor of size 1x10]
```

一个真实世界的例子

> *现在我们考虑一个使用IWSLT德英的真实例子 翻译任务。该任务远小于WMT任务，涉及 但这说明了整个系统。我们还展示了如何使用多显卡 处理速度让它非常快。*

```
#!pip install torchtext spacy
#!python -m spacy download en
#!python -m spacy download de
```

数据加载

> *我们将使用 torchtext 和 spacy 进行代币化加载数据集。*

```
# For data loading.
from torchtext import data, datasets

if True:
    import spacy
    spacy_de = spacy.load('de')
    spacy_en = spacy.load('en')

    def tokenize_de(text):
        return [tok.text for tok in spacy_de.tokenizer(text)]

    def tokenize_en(text):
        return [tok.text for tok in spacy_en.tokenizer(text)]

    BOS_WORD = '<s>'
    EOS_WORD = '</s>'
    BLANK_WORD = "<blank>"
    SRC = data.Field(tokenize=tokenize_de, pad_token=BLANK_WORD)
    TGT = data.Field(tokenize=tokenize_en, init_token = BOS_WORD,
                     eos_token = EOS_WORD, pad_token=BLANK_WORD)

    MAX_LEN = 100
    train, val, test = datasets.IWSLT.splits(
        exts=('.de', '.en'), fields=(SRC, TGT),
        filter_pred=lambda x: len(vars(x)['src']) <= MAX_LEN and
            len(vars(x)['trg']) <= MAX_LEN)
    MIN_FREQ = 2
    SRC.build_vocab(train.src, min_freq=MIN_FREQ)
    TGT.build_vocab(train.trg, min_freq=MIN_FREQ)
```

> *批量处理对速度非常重要。我们希望分批非常均匀， 几乎没有填充物。为此，我们必须对 默认的Torchtext批处理。这段代码会补丁它们默认的批处理 当然，我们会搜索足够多的句子以找到紧凑的批次。*

迭代器

```
class MyIterator(data.Iterator):
    def create_batches(self):
        if self.train:
            def pool(d, random_shuffler):
                for p in data.batch(d, self.batch_size * 100):
                    p_batch = data.batch(
                        sorted(p, key=self.sort_key),
                        self.batch_size, self.batch_size_fn)
                    for b in random_shuffler(list(p_batch)):
                        yield b
            self.batches = pool(self.data(), self.random_shuffler)

        else:
            self.batches = []
            for b in data.batch(self.data(), self.batch_size,
                                         self.batch_size_fn):
                self.batches.append(sorted(b, key=self.sort_key))

def rebatch(pad_idx, batch):
    "Fix order in torchtext to match ours"
    src, trg = batch.src.transpose(0, 1), batch.trg.transpose(0, 1)
    return Batch(src, trg, pad_idx)
```

多GPU训练

> *最后，为了真正针对快速训练，我们将使用多显卡。这套代码 实现多GPU字生成。这不是特有的变压器，所以我 我就不详细说了。其思路是将词生成拆分于 将训练时间分解为块，并行处理多个不同的区域 GPU。我们使用pytorch并行原语实现这一点：*

- 复制——将模块拆分到不同的显卡上。
- Scatter - 将批次拆分到不同的显卡上
- parallel_apply - 将模块应用到不同GPU的批量
- 收集——将分散的数据拉回一块显卡。
- 嗯。DataParallel——一个特殊的模块包装器，在之前调用所有这些 评估。

```
# Skip if not interested in multigpu.
class MultiGPULossCompute:
    "A multi-gpu loss compute and train function."
    def __init__(self, generator, criterion, devices, opt=None, chunk_size=5):
        # Send out to different gpus.
        self.generator = generator
        self.criterion = nn.parallel.replicate(criterion,
                                               devices=devices)
        self.opt = opt
        self.devices = devices
        self.chunk_size = chunk_size

    def __call__(self, out, targets, normalize):
        total = 0.0
        generator = nn.parallel.replicate(self.generator,
                                                devices=self.devices)
        out_scatter = nn.parallel.scatter(out,
                                          target_gpus=self.devices)
        out_grad = [[] for _ in out_scatter]
        targets = nn.parallel.scatter(targets,
                                      target_gpus=self.devices)

        # Divide generating into chunks.
        chunk_size = self.chunk_size
        for i in range(0, out_scatter[0].size(1), chunk_size):
            # Predict distributions
            out_column = [[Variable(o[:, i:i+chunk_size].data,
                                    requires_grad=self.opt is not None)]
                          for o in out_scatter]
            gen = nn.parallel.parallel_apply(generator, out_column)

            # Compute loss.
            y = [(g.contiguous().view(-1, g.size(-1)),
                  t[:, i:i+chunk_size].contiguous().view(-1))
                 for g, t in zip(gen, targets)]
            loss = nn.parallel.parallel_apply(self.criterion, y)

            # Sum and normalize loss
            l = nn.parallel.gather(loss,
                                   target_device=self.devices[0])
            l = l.sum()[0] / normalize
            total += l.data[0]

            # Backprop loss to output of transformer
            if self.opt is not None:
                l.backward()
                for j, l in enumerate(loss):
                    out_grad[j].append(out_column[j][0].grad.data.clone())

        # Backprop all loss through transformer.
        if self.opt is not None:
            out_grad = [Variable(torch.cat(og, dim=1)) for og in out_grad]
            o1 = out
            o2 = nn.parallel.gather(out_grad,
                                    target_device=self.devices[0])
```

```
            o1.backward(gradient=o2)
            self.opt.step()
            self.opt.optimizer.zero_grad()
        return total * normalize
```

> *现在我们创建模型、准则、优化器、数据迭代器和 平行化*

```
# GPUs to use
devices = [0, 1, 2, 3]
if True:
    pad_idx = TGT.vocab.stoi["<blank>"]
    model = make_model(len(SRC.vocab), len(TGT.vocab), N=6)
    model.cuda()
    criterion = LabelSmoothing(size=len(TGT.vocab), padding_idx=pad_idx, smoothing=0.1)
    criterion.cuda()
    BATCH_SIZE = 12000
    train_iter = MyIterator(train, batch_size=BATCH_SIZE, device=0,
                            repeat=False, sort_key=lambda x: (len(x.src), len(x.trg)),
                            batch_size_fn=batch_size_fn, train=True)
    valid_iter = MyIterator(val, batch_size=BATCH_SIZE, device=0,
                            repeat=False, sort_key=lambda x: (len(x.src), len(x.trg)),
                            batch_size_fn=batch_size_fn, train=False)
    model_par = nn.DataParallel(model, device_ids=devices)
None
```

> *现在我们训练模型。我会稍微调整热身步骤，但 其他的都用默认参数。在一台配备4台特斯拉的AWS p3.8xlarge上 V100s，这以每秒约27,000个令牌运行，批次大小为12,000*

系统培训

```
#!wget https://s3.amazonaws.com/opennmt-models/iwslt.pt (https://s3.amazonaws.com/opennmt-models/iwslt.pt)
```

```
if False:
    model_opt = NoamOpt(model.src_embed[0].d_model, 1, 2000,
            torch.optim.Adam(model.parameters(), lr=0, betas=(0.9, 0.98), eps=1e-9))
    for epoch in range(10):
        model_par.train()
        run_epoch((rebatch(pad_idx, b) for b in train_iter),
                  model_par,
                  MultiGPULossCompute(model.generator, criterion,
                                      devices=devices, opt=model_opt))
        model_par.eval()
        loss = run_epoch((rebatch(pad_idx, b) for b in valid_iter),
                         model_par,
                         MultiGPULossCompute(model.generator, criterion,
                         devices=devices, opt=None))
        print(loss)
else:
    model = torch.load("iwslt.pt")
```

> *训练完成后，我们可以解码模型，生成一组翻译。我们只需翻译验证集中的第一句话即可。该数据集为 规模相当小，所以用贪婪搜索的翻译还算准确。*

```
for i, batch in enumerate(valid_iter):
    src = batch.src.transpose(0, 1)[:1]
    src_mask = (src != SRC.vocab.stoi["<blank>"]).unsqueeze(-2)
    out = greedy_decode(model, src, src_mask,
                        max_len=60, start_symbol=TGT.vocab.stoi["<s>"])
    print("Translation:", end="\t")
    for i in range(1, out.size(1)):
        sym = TGT.vocab.itos[out[0, i]]
        if sym == "</s>": break
        print(sym, end =" ")
    print()
    print("Target:", end="\t")
    for i in range(1, batch.trg.size(0)):
        sym = TGT.vocab.itos[batch.trg.data[i, 0]]
        if sym == "</s>": break
        print(sym, end =" ")
    print()
    break
```

```
Translation:    <unk> <unk> . In my language , that means , thank you very much .
Gold:   <unk> <unk> . It means in my language , thank you very much .
```

附加组件：BPE、搜索、平均

> *所以这主要涵盖了变压器模型本身。主要有四个方面 而我们并没有明确讨论。我们还有这些额外的功能 在 OpenNMT-py (https://github.com/opennmt/opennmt-py) 中实现。*

> *1） BPE/Word-piece：我们可以先用库将数据预处理成 子词单位。参见里科·森里奇的副词—— 核磁性测距 (https://github.com/rsennrich/subword-nmt)实现。这些型号会将训练数据转换为如下样式：*

Die Protokoll datei kann heimlich per E - Mail oder FTP an einen bestimmte n Empfänger gesendet werden .

> *2）共享嵌入：当使用带有共享词汇的BPE时，我们可以共享 源/目标/发生器之间的权重矢量相同。详情请参见（引文） (https://arxiv.org/abs/1608.05859)条目。将此添加到模型中 只需这样做：*

```
if False:
    model.src_embed[0].lut.weight = model.tgt_embeddings[0].lut.weight
    model.generator.lut.weight = model.tgt_embed[0].lut.weight
```

> *3）光束搜索：这部分太复杂，无法在这里详述。参见OpenNMT—— Py (https://github.com/OpenNMT/OpenNMT-py/blob/master/onmt/translate/Beam.py) 用于 Pytorch 实现。*

> *4）模型平均：论文对最近k个检查点进行平均，以建立 合奏效果。如果我们有一堆模型，事后可以做到：*

```
def average(model, models):
    "Average models into model"
    for ps in zip(*[m.params() for m in [model] + models]):
        p[0].copy_(torch.sum(*ps[1:]) / len(ps[1:]))
```

选举结果

在WMT 2014英德翻译任务中，大变压器模型 （变压金刚（大） 在表2中）优于此前报道的最佳模型（包括合奏）的2.0多点 BLEU，建立了新的先进BLEU分数28.4。配置 该模型的 列于表3底部。训练用了3.5天，使用8块P100显卡。 甚至是我们的基础模型 超越了所有此前已发表的模型和集合，仅为 任何训练费用 竞争型。

在WMT 2014英法翻译任务中，我们的大模型实现了 BLEU 评分为41.0， 超过所有先前发布的单一模型，性能不足1/4 培训费用 之前的先进型号。训练为 的变换器（大型）模型 英语到法语的使用情况 掉落率 Pdrop = 0.1，而非 0.3。

```
Image(filename="images/results.png")
```

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

我们写的代码是基础模型的一个版本。完全是 该系统的训练版本可在此获取（示例） 模特） (http://opennmt.net/Models-py/)。

在最后一节的扩展中，OpenNMT-py 复制 在EN-DE WMT上达到26.9。这里我已将这些参数加载到我们的 重新实施。

```
!wget https://s3.amazonaws.com/opennmt-models/en-de-model.pt
```

```
model, SRC, TGT = torch.load("en-de-model.pt")
```

```
model.eval()
sent = "__The __log __file __can __be __sent __secret ly __with __email __or __FTP __to
__a __specified __receiver".split()
src = torch.LongTensor([[SRC.stoi[w] for w in sent]])
src = Variable(src)
src_mask = (src != SRC.stoi["<blank>"]).unsqueeze(-2)
out = greedy_decode(model, src, src_mask,
                    max_len=60, start_symbol=TGT.stoi["<s>"])
print("Translation:", end="\t")
trans = "<s> "
for i in range(1, out.size(1)):
    sym = TGT.itos[out[0, i]]
    if sym == "</s>": break
    trans += sym + " "
print(trans)
```

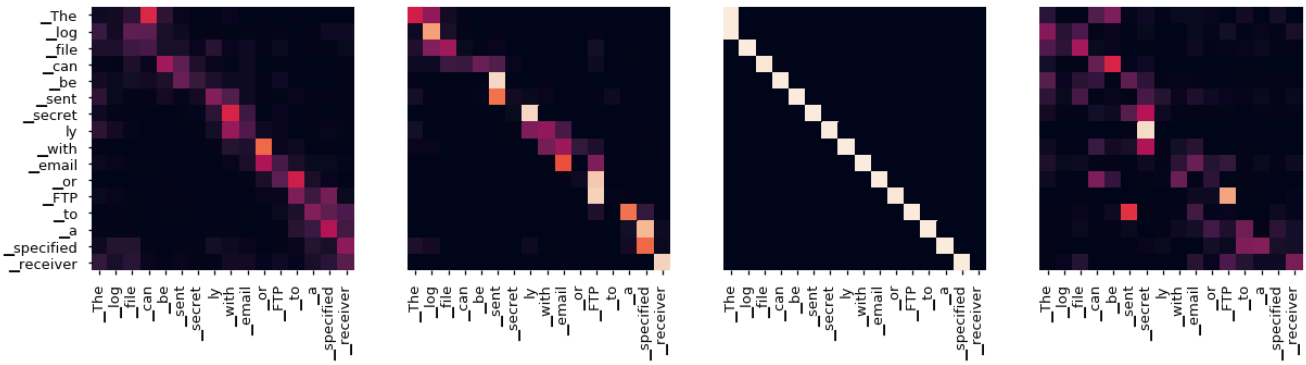
```
Translation:    <s> __Die __Protokoll datei __kann __ heimlich __per __E - Mail __oder __
FTP __an __einen __bestimmte n __Empfänger __gesendet __werden .
```

注意力可视化

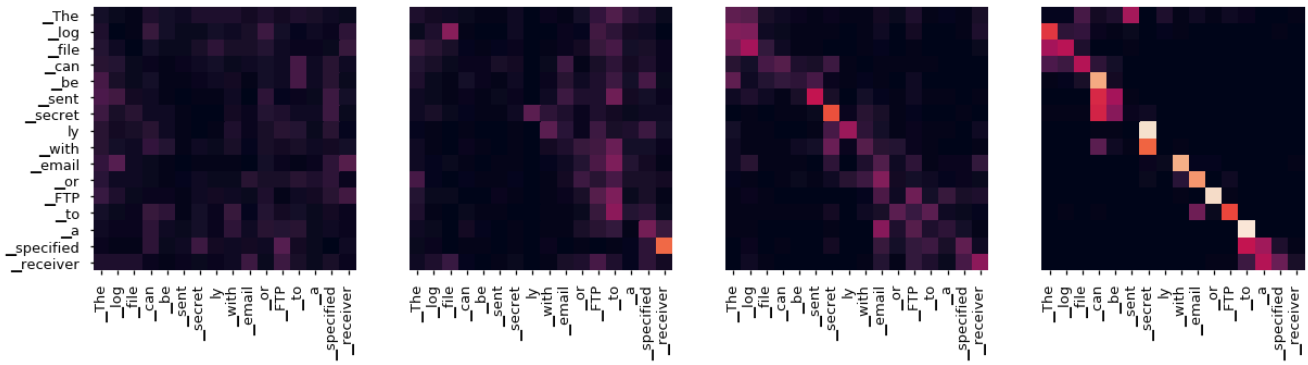
即使用了贪婪的解码器，翻译看起来也相当不错。我们可以进一步发展 想象一下，看看注意力的每一层发生了什么

```
tgt_sent = trans.split()
def draw(data, x, y, ax):
    seaborn.heatmap(data,
                    xticklabels=x, square=True, yticklabels=y, vmin=0.0, vmax=1.0,
                    cbar=False, ax=ax)

for layer in range(1, 6, 2):
    fig, axs = plt.subplots(1,4, figsize=(20, 10))
    print("Encoder Layer", layer+1)
    for h in range(4):
        draw(model.encoder.layers[layer].self_attn.attn[0, h].data,
            sent, sent if h ==0 else [], ax=axs[h])
    plt.show()

for layer in range(1, 6, 2):
    fig, axs = plt.subplots(1,4, figsize=(20, 10))
    print("Decoder Self Layer", layer+1)
    for h in range(4):
        draw(model.decoder.layers[layer].self_attn.attn[0, h].data[:len(tgt_sent), :len(t
gt_sent)],
            tgt_sent, tgt_sent if h ==0 else [], ax=axs[h])
    plt.show()
    print("Decoder Src Layer", layer+1)
    fig, axs = plt.subplots(1,4, figsize=(20, 10))
    for h in range(4):
        draw(model.decoder.layers[layer].self_attn.attn[0, h].data[:len(tgt_sent), :len(s
ent)],
            sent, tgt_sent if h ==0 else [], ax=axs[h])
    plt.show()
```

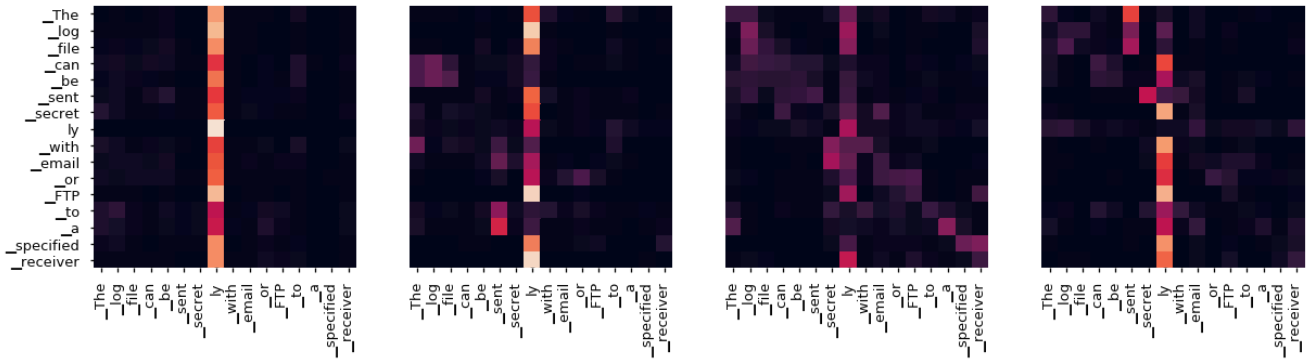
```
Encoder Layer 2
```



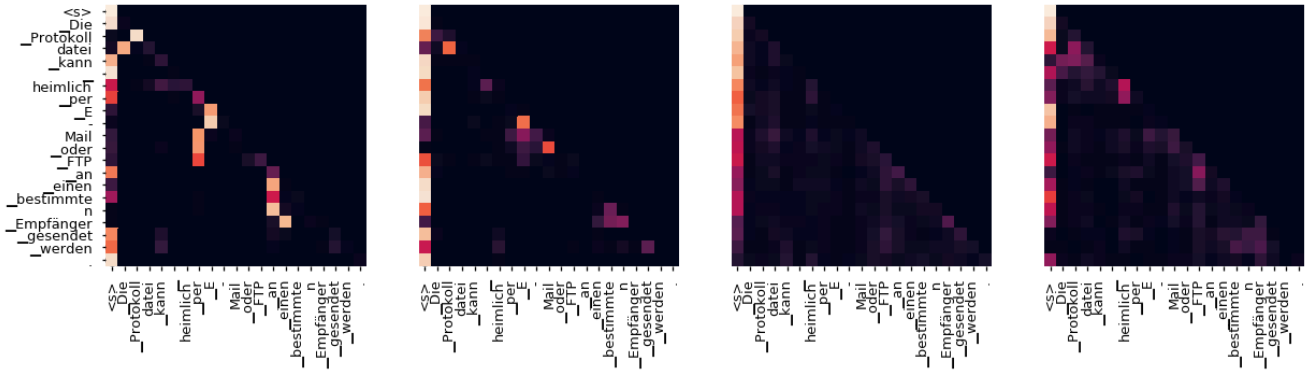

```
Encoder Layer 4
```



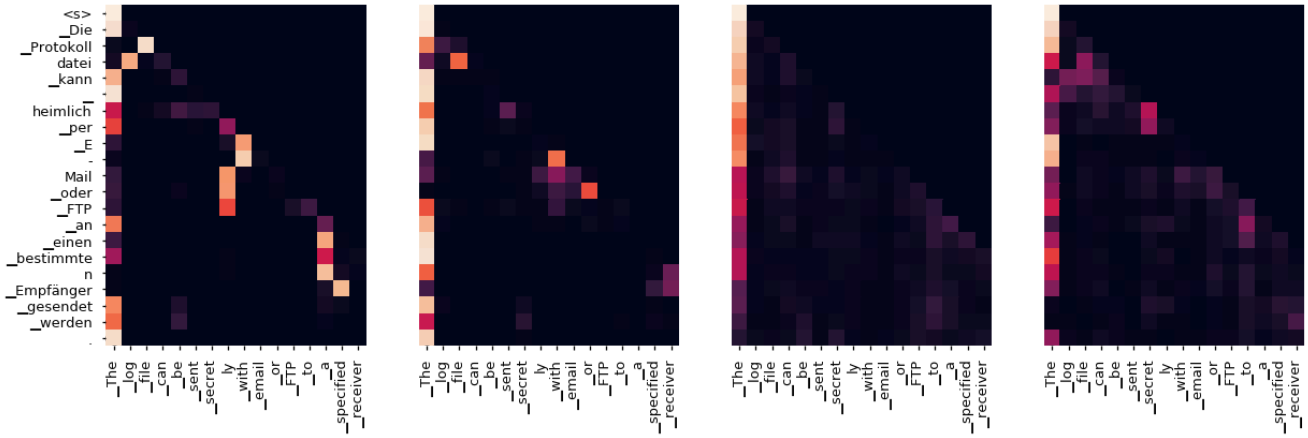
Encoder Layer 6



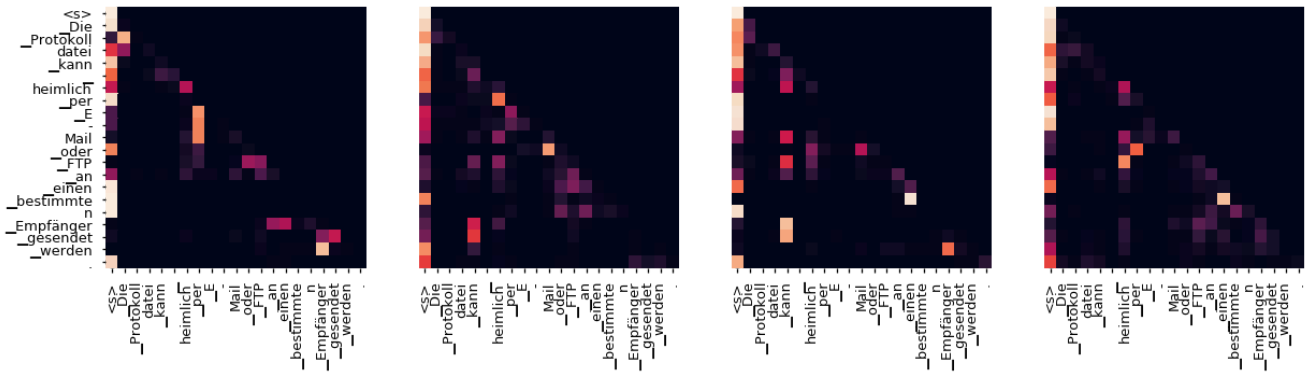
Decoder Self Layer 2



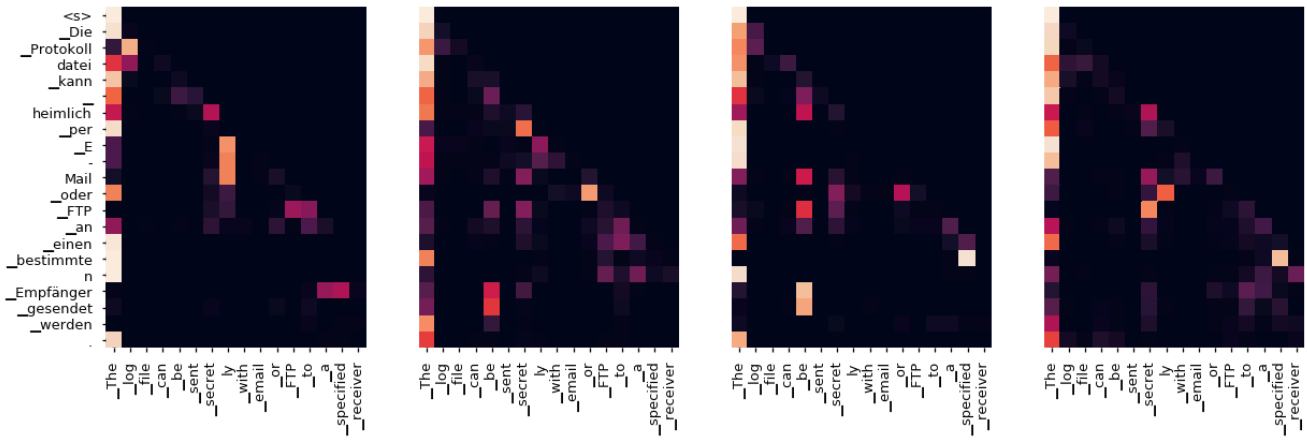
Decoder Src Layer 2



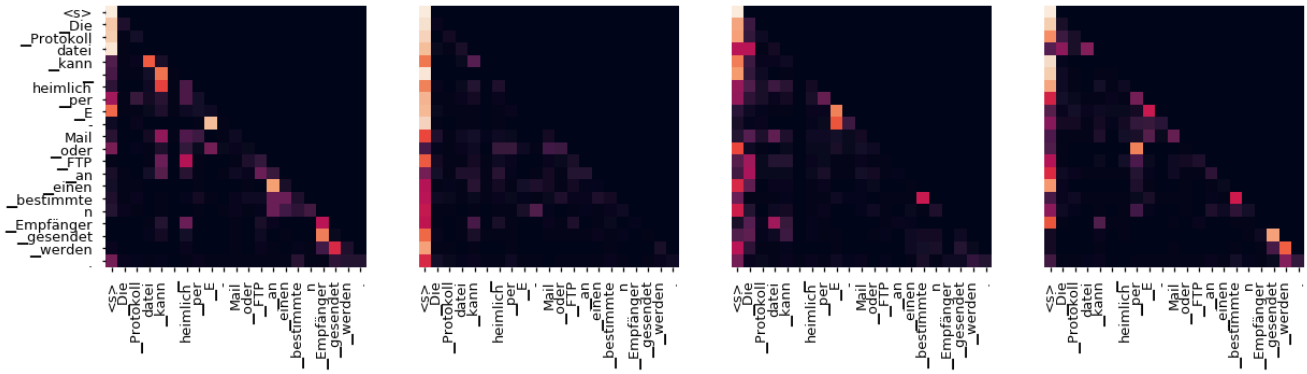
Decoder Self Layer 4



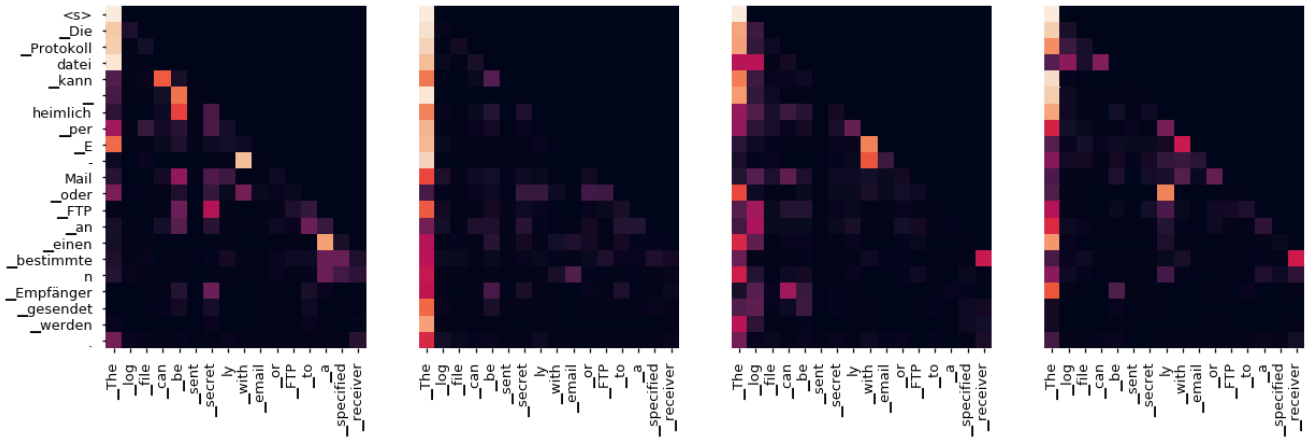
Decoder Src Layer 4



Decoder Self Layer 6



Decoder Src Layer 6



结论

> *希望这段代码对未来的研究有所帮助。如果你有任何问题，请联系我。如果你觉得这段代码有帮助，也请查看我们的其他OpenNMT工具。*

```
@inproceedings{opennmt,
  author    = {Guillaume Klein and
               Yoon Kim and
               Yuntian Deng and
               Jean Senellart and
               Alexander M. Rush},
  title     = {OpenNMT: Open-Source Toolkit for Neural Machine Translation},
  booktitle = {Proc. ACL},
  year      = {2017},
  url       = {https://doi.org/10.18653/v1/P17-4012 (https://doi.org/10.18653/v1/P17-401
2)},
  doi       = {10.18653/v1/P17-4012}
}
```

> *干杯，Srush*